

CSC144V Lab 3: Lane Detection with UFLD and Adversarial Attacks

Algorithms and Systems for Autonomous Vehicles, Hofstra University, Fall 2026. Instructor: Zonghua Gu. Work in your groups, as in Lab 2. Due date: see the course website.

1. Overview

In this lab you work with a pretrained, real-time lane detector, Ultra-Fast-Lane-Detection (UFLD v1, ResNet-18). You will not train a network. Instead you (1) run the detector and measure what it can and cannot do, (2) turn its output into a lane departure warning, using the same cross-track-error idea as in Lab 2, and (3) attack it with the gradient-based methods from Lab 1 and study how a small image perturbation changes the vehicle's estimated position in its lane.

Prerequisites: L3 CNN, L5 Object Detection and Segmentation, L4.1 and L4.2 Adversarial Attacks, L8 Control, Lab 1 and Lab 2.

2. Background: how UFLD detects lanes

Most lane detectors classify every pixel (segmentation), which is slow. UFLD instead treats lane detection as row-wise classification: the image is resized to 288 x 800, 56 horizontal row anchors are placed on the road region, and for each (row anchor, lane) the network chooses one of $G = 100$ grid columns where the lane crosses that row, or class $G = \text{"no lane"}$. Up to 4 lanes are predicted, ordered left to right.

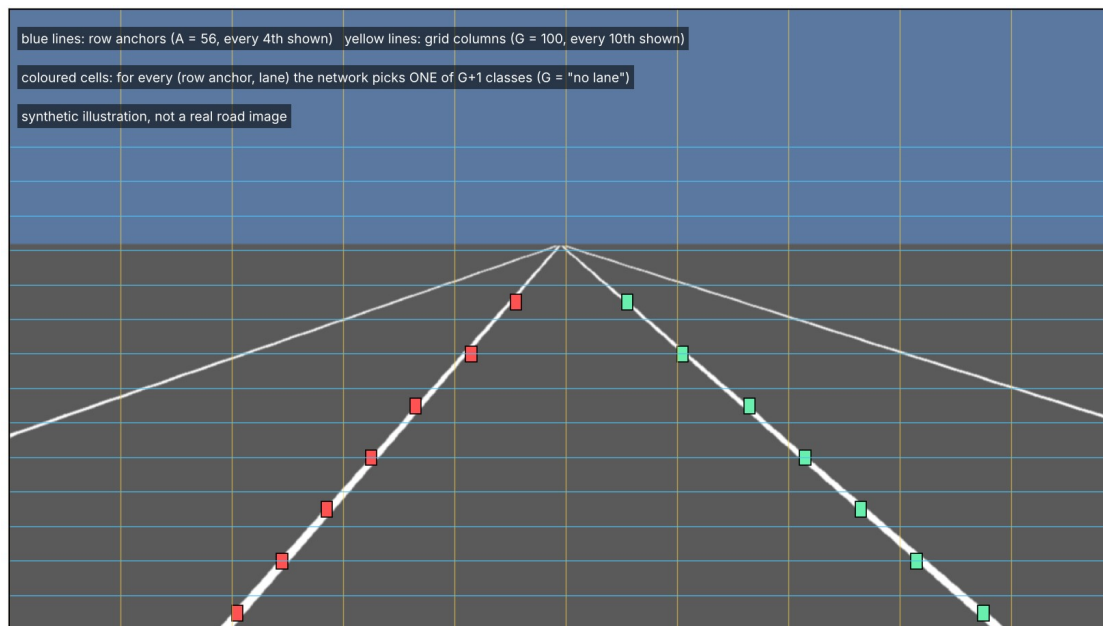


Figure 1. Row-anchor classification (illustration on a synthetic road).

Quantity	Value (TuSimple model)
Network input	RGB image resized to 288 x 800, ImageNet-normalised
Network output (logits)	tensor of shape (1, 101, 56, 4) = ($G+1$ classes, A row anchors, 4 lane slots)
Grid cell width	$799 / 99 = 8.07$ px in the 800-px-wide network input (about 12.9 px in a 1280-px TuSimple frame)

Quantity	Value (TuSimple model)
Decoding	softmax over the G columns, expected column per (anchor, lane); a lane needs more than 2 valid points
Speed reported by the authors	about 300 FPS on a GTX 1080Ti (ResNet-18)

Because the output is a classification, cross-entropy and FGSM/PGD from Lab 1 apply with almost no change. The class label of a cell is just its grid column.

3. Setup

Use Google Colab with a GPU runtime. Tasks 1 and 2 also run on CPU, but Task 3 needs many forward and backward passes and is slow on CPU. The provided helper module `ufl_d_lab.py` wraps the UFLD repository so that it runs on any image or video, on CPU or GPU, and with gradients with respect to the input. Do not modify it.

```
!git clone https://github.com/cfzd/Ultra-Fast-Lane-Detection
!pip install gdown
!gdown 1WCYyur5ZaWczH15ecmeDowrW30xcLrCn -O tusimple_18.pth # TuSimple weights
from the repo README
# upload ufl_d_lab.py and lab3_data.zip (frames/ and videos/) from Canvas, then:
from ufl_d_lab import LaneDetector, draw_lanes, video_frames, cell_stats
det = LaneDetector('Ultra-Fast-Lane-Detection', 'tusimple_18.pth', 'Tusimple')
```

If Google Drive refuses the download (quota), use the copy of the weights posted on Canvas. A deprecation warning about torchvision's pretrained argument is harmless.

Data (lab3_data.zip on Canvas): a set of road frames and a few short driving videos. You may also add your own dashcam video for Task 2 (optional).

4. Task 1: Run the pretrained detector (20 points)

- Run the detector on all provided frames. Show at least 6 results (lanes drawn on the image) in a grid.
- Show 2 failure cases (missing lane, extra lane, wrong slot, jitter, ...) and explain the likely cause.
- Measure inference time per frame over 100 runs, split into pre-processing, network and decoding, on CPU and (if available) GPU. Report milliseconds and FPS.
- Answer briefly: (a) what do the four dimensions of the output tensor mean? (b) why is row-anchor classification cheaper than per-pixel segmentation? (c) what does class G mean and how is it used when decoding? (d) how many pixels wide is one grid cell in the original frame?

5. Task 2: Lane departure warning (35 points)

Goal: estimate how far the vehicle is from the centre of its lane, as a lateral offset in metres, and raise a warning when the vehicle drifts too far. This is the cross-track error (cte) that the PID controller in Lab 2 needs.

5.1 Implement `lateral_offset` (15 points)

Write `lateral_offset(lanes, img_w, img_h, y_ref=0.9, lane_width_m=3.7, deg=2)` with the following specification:

- Input: the list of 4 lane arrays returned by `det.decode / det.detect` (each of shape $(n, 2)$ with x, y pixels; empty if the slot has no lane).
- Fit $x = f(y)$ with a polynomial of degree `deg` to each lane with at least 3 points, and evaluate it at the row $y = y_ref * img_h$.
- Ego lane: the closest lane line to the left of the image centre and the closest to the right. Do not simply use slots 1 and 2, because slots can be empty or shifted. Return `None` if either side is missing.
- Assume the camera sits at the vehicle centre, so the vehicle position is $x = img_w / 2$. Define $offset_px = img_w / 2 - (x_left + x_right) / 2$ (positive means the vehicle is to the right of the lane centre).
- Convert to metres with the lane width in pixels at that row: $offset_m = offset_px * lane_width_m / (x_right - x_left)$.

Test your function on a few frames and show that the sign and magnitude make sense (for example by drawing the two ego lines and the lane centre).

5.2 Time series and warning (20 points)

- Run the detector on a provided video and plot `offset_m` versus time. Mark frames where the ego lane is lost.
- Smooth with an exponential moving average, with α in $\{0.1, 0.3, 0.5\}$. Plot raw and smoothed signals together.
- Warning rule: warn when $|offset_m|$ exceeds 0.6 m for at least 3 consecutive frames. Report the number of warning events and show 3 frames at which a warning is triggered.
- Quantify jitter as the standard deviation of the frame-to-frame difference, raw versus smoothed, and discuss the delay (lag) that smoothing introduces.
- Briefly discuss: which assumptions does your estimate rely on (camera at vehicle centre, flat road, 3.7 m lane width, correct ego-lane selection), and when do they fail?

6. Task 3: Adversarial attacks on the lane detector (40 points)

Threat model. White-box and digital: you add a perturbation to the 288×800 network input x in $[0, 1]$, with the constraint that the L-infinity norm is at most ϵ (pixel values are in $[0, 1]$, so $\epsilon = 8/255$ means 8 grey levels). No ground truth is needed: use the model's own clean prediction $y_clean = \text{argmax}$ of the clean logits over the class dimension (shape $(1, 56, 4)$) as the label. Work offline on the provided frames only.

Useful building blocks: `det.to_tensor(img)` gives x ; `det.forward(x)` is differentiable (`det.logits` is not); `F.cross_entropy(logits, y)` accepts logits of shape $(1, 101, 56, 4)$ and labels of shape $(1, 56, 4)$ directly; `cell_stats(det, logits_clean, logits_adv)` summarises the effect on grid cells.

6.1 FGSM, untargeted (10 points)

Implement `fgsm(det, x, y, eps)` and run it for ϵ in $\{2, 4, 8, 16\}/255$.

6.2 PGD, untargeted (10 points)

Implement `pgd_linf(det, x, y, eps, alpha, steps)` (10 to 20 steps, α around $\epsilon / 4$), projecting onto the ϵ -ball and onto $[0, 1]$ after every step. Compare with FGSM at the same ϵ .

6.3 PGD, targeted lane shift (15 points)

Build a target label that moves every lane cell s grid columns to the right: $y_target = y_clean + s$ where $y_clean < G$, and no-lane cells stay unchanged. Run PGD that minimises the cross-entropy to

y_{target} . Try s in $\{+2, +4, +6\}$ (one cell is about 8 px) and, if needed, more steps (up to 100) and larger eps. Report the fraction of cells that reach the target and the mean shift.

Effect on the vehicle estimate. For every attack, feed the decoded lanes to your `lateral_offset` (decode at network resolution with `det.lanes_at_net_res`; the offset in metres does not depend on the resolution) and report the change in `offset_m` and whether the lane departure warning changes (false warning, or a hidden warning).

6.4 Report and discussion (5 points)

Use at least 10 different frames and fill in a table like the one below (averages over frames). Show for one frame: original, amplified perturbation, attacked frame, with detected lanes drawn on both.

eps	Attack	Cells changed (%)	Mean shift (px)	Mean change in offset (m)
2/255	FGSM			
8/255	PGD untargeted			
8/255	PGD targeted, $s = +4$			
...	...			

Discussion (a few sentences each): (a) which attack is the most effective per unit of eps, and why? (b) a targeted shift moves the estimated lane centre; what would a lane-keeping controller do with this, and in which direction? (c) why is this digital attack not directly a physical-world attack, and what would an attacker need in addition (printable patch, robustness to viewpoint and lighting, ...)? (d) name one defence and say what you expect it to do to the table above (for example JPEG compression or blurring of the input).

Hints. Keep the perturbed image inside $[0, 1]$ (clamp). Targeted attacks descend the loss, untargeted attacks ascend it. Targeted shifts are harder than untargeted attacks and may need more steps or a larger eps, and a failed attempt is a valid result if you report it honestly together with the settings you tried. Do not wrap attack code in `torch.no_grad`.

7. Submission

- A Colab link (sharing set to anyone with the link) or a Lab3-GroupName.ipynb file, executed so that all outputs are visible.
- A short report in PDF (at most 6 pages) with the figures, tables and written answers of Tasks 1 to 3, as well as any problems you encountered or insights you want to share. You may embed the report in the notebook and print it as PDF.

8. Grading

Part	Points
Task 1: run detector, failure cases, timing, short answers	20
Task 2: <code>lateral_offset</code> (15), time series and warning (20)	35
Task 3: FGSM (10), PGD untargeted (10), targeted shift (15), report and discussion (5)	40
Clarity of the report and code	5
Total	100

Ethics. The attacks in this lab are for understanding the robustness of perception systems. Use them only on the provided images and your own offline data, never on a vehicle or on public roads.

References

- Z. Qin, H. Wang, X. Li, Ultra Fast Structure-aware Deep Lane Detection, ECCV 2020. Code: github.com/cfzd/Ultra-Fast-Lane-Detection (MIT license).
- I. Goodfellow, J. Shlens, C. Szegedy, Explaining and Harnessing Adversarial Examples, ICLR 2015 (FGSM).
- A. Madry et al., Towards Deep Learning Models Resistant to Adversarial Attacks, ICLR 2018 (PGD).
- Adversarial Robustness: Theory and Practice, NeurIPS 2018 tutorial (used in Lab 1): adversarial-ml-tutorial.org.
- TuSimple lane detection benchmark (images and labels used to train the provided weights).